

Product Requirements

What we're building, for whom, and how we'll know it works.

Read this if: everyone — product, engineering, design

The problem

Most code tools tell you what a function does. None of them tell you why it exists, who to ask, or what's about to break if the person who understands it leaves. Code Archaeologist is built to close that gap for a public GitHub repository.

Who it's for

- Developers joining a codebase they've never seen
- Team members moving into an unfamiliar module
- Senior engineers, tech leads, and engineering managers
- Maintainers of open-source projects

The user journey

Sign in → Add a public repository → Run analysis → Browse structure → Get task guidance → Explore code and history → Ask "why?" → Get an evidence-backed answer → Review flagged risks → Hit a knowledge gap → Provide the human answer → That answer gets reused automatically → Run offboarding before a contributor leaves

Functional requirements

ID	Area	Requirement
FR-01	Auth	Sign up, log in, log out
FR-02	Isolation	Users only see their own repositories, analyses, and knowledge
FR-03	Import	Submit a public GitHub repository URL
FR-04	Acquisition	Repository is pulled for static analysis only — never executed
FR-05	Analysis	Extract structure, history, contributors, code entities, and relationships
FR-06	Languages	Python, JavaScript, TypeScript, Java, C++, Go at launch; graceful degradation elsewhere
FR-07	Explorer	Browse repository files
FR-08	Code viewer	View source via Monaco Editor
FR-09	Architecture view	See module and dependency relationships
FR-10	History	Inspect commit and history context
FR-11	Historian	Ask repository-specific "why" questions
FR-12	Evidence	Every answer shows its evidence and classification
FR-13	Confidence	Answers labeled High / Medium / Low
FR-14	Dev guidance	Describe a task, get a prioritized path through the repo
FR-15	Risk	Flag fragile areas using deterministic signals, not an LLM guess
FR-16	Knowledge gaps	Recognize when evidence isn't enough to answer confidently
FR-17	Human knowledge	Users supply an explanation; it's stored and reused
FR-18	Offboarding	Identify at-risk knowledge and generate transfer questions and a report
FR-19	Guardrails	Reject off-topic questions; treat repository content as untrusted, never as instructions

Non-functional requirements

- Analysis is strictly read-only — no code execution, ever
- Secrets never reach the client
- User data is isolated per account
- Errors are clear and actionable, not silent
- Module boundaries stay clean for parallel team work
- API contracts are shared and respected
- Unsupported languages degrade gracefully
- Both frontend and backend are actually deployed, not just runnable locally

What success looks like

A judge (or a new hire) should be able to follow one unbroken chain: repository history → why a decision was made → evidence-backed answer → a flagged risk or knowledge gap → a human fills the gap → that knowledge gets reused → it's preserved through offboarding.

Scope boundary

Code Archaeologist answers questions about the repository it's analyzing. It politely declines anything else.